

Agent 架构认知手册

从三个热点出发,一篇博客的深度解剖,以及五条可带走的判断

本手册沉淀一段关于 2026 年 Agent 领域的认知讨论。它从对 Harness、Self-Evolution、Agentic Memory 三个热点的校准出发,经由 LangChain 三层模型与 L1-L5 框架的对照,最终以 Databricks "Memory Scaling" 这篇博客为深度解剖样本,梳理出适用于企业级 Agent 平台架构的判断与方法。

阅读路径

5 分钟	读 Part 1 · 拿到 2026 Agent 的全景立场
30 分钟	读 Part 1 + 2 · 拿到可套用的核心框架
1 小时	加 Part 3 · 用 Memory Scaling 作为完整解剖样本下钻
引用资料	跳到 Part 4 + 5 · 判断清单 / 引用索引

PART 1

立场与全景

如果只读三页,这一部分回答一个问题:2026 年的 Agent 领域到底在发生什么?

1.1 一句话立场

CORE THESIS

基础模型会趋同,基础模型之上的 scaffolding 形态不会。Memory 是唯一一条红利归属于企业自己、而非基础模型提供方的 scaling 轴。

这个立场建立在三条相互支撑的判断上:

判断 A

判断 B

判断 C

三条 scaling 轴

Parametric(模型做大)·
Inference-time(推理变深)·
Memory(外部状态累积)三
轴并列。前两条归属基础设
施提供方,第三条归属企业自
己。

Scaffolding 跨升级

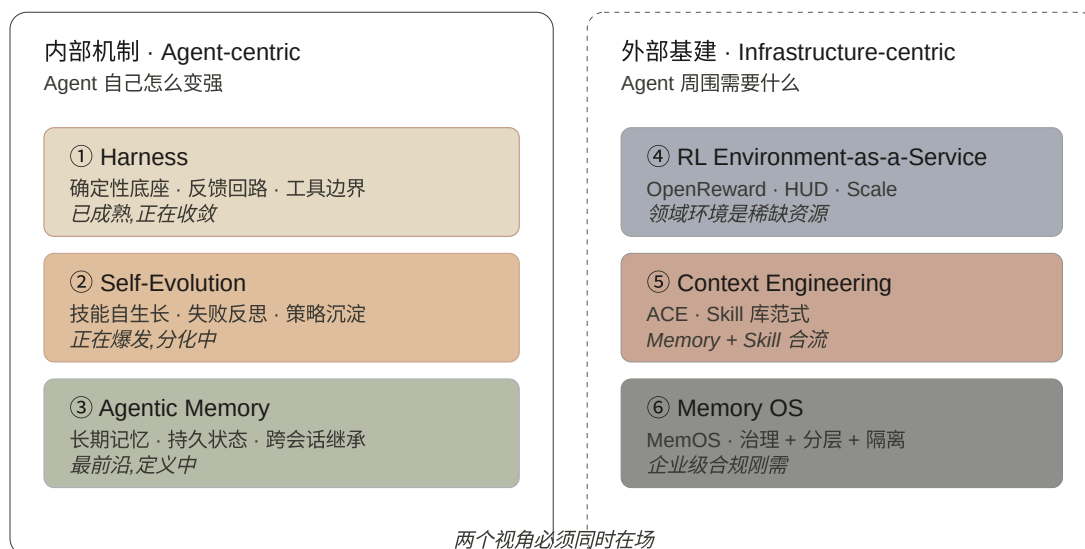
Skill / workflow / memory
是模型无关的;weights-
bound 改进会被下一代模型
抹平。把投入放在
scaffolding 层,跨越每一次
模型升级。

领域是真正护城河

框架熟练度不是壁垒。领域
工具质量、记忆架构设计、
知识编码、数据飞轮密度才
是,且都不能被代码或架构泄
漏复制。

1.2 六概念地图

本次讨论的起点是用户提出的三个"内部机制"概念:Harness、Self-Evolution、
Agentic Memory。但 2026 年真正在重构格局的,还有三个被忽视的"外部基建"概念。
两组合在一起才是完整地图。



内部机制 vs 外部基建 · 两个视角必须同时在场

内部三热点

- **Harness** — 让 AI 在确定性下工作。已成熟,正在收敛。OpenAI Codex 用此方法写了 100 万+行代码,AGENTS.md 成为事实标准。
- **Self-Evolution** — 让 AI 自己成长、沉淀、修正。正在爆发,分化中。ACE 论文 (ICLR 2026)是关键节点,Skill 库范式在多个项目中确立。
- **Agentic Memory** — 怎么真正组织 AI 的长期记忆。最前沿,定义中。Databricks 给出首条可量化 scaling 曲线,LOCOMO 成为首个跨架构评测基准。

外部三基建

- **RL Environment-as-a-Service** — 训练环境作为托管 API。OpenReward(扩展 MCP 的 ORS 协议)、HUD、Scale RL Environments。领域环境本身正在成为稀缺资源。
- **Context Engineering** — 把"管理上下文窗口"独立成方法论。Memory + Skill 在这个学科下合流。北大 Meta Context Engineering 是代表工作。
- **Memory OS** — 把记忆治理作为操作系统级基础设施。MemOS 提出分层记忆 + ACL + 版本化的完整治理框架。

1.3 这六概念为什么必须一起看

只看内部三个 → 会陷入"Agent 自我演化"的乌托邦,忽视基建才是真护城河。

只看外部三个 → 会变成空中楼阁,看不到 Agent 自己的成长机制在哪里发挥。

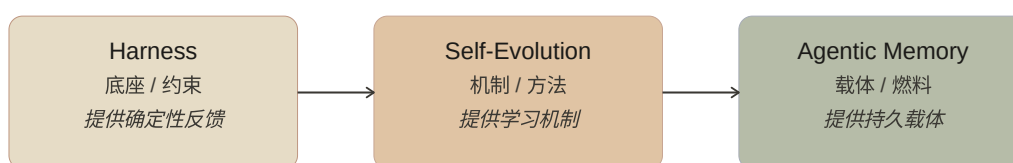
对任何做 Agent 平台的团队,两个视角必须同时在场:内部回答"Agent 自己怎么变强",外部回答"Agent 周围需要什么基建"。任何一边的缺失都会让架构判断系统性偏移。

核心框架

如果你要做架构决策,这一部分给两个可直接套用的框架。它们不是替代关系,而是从不同角度切同一个问题。

2.1 三热点的依赖链

第一个框架很简单:三热点不是并列概念,而是有严格依赖顺序。



没有 Harness 的反馈,Self-Evolution 学到的都是噪声
没有 Memory 的载体,演化结果随会话消失
这不是并列,是严格的依赖顺序

底座 → 机制 → 燃料 · 反过来做一定失败

这条链的实战含义:先有 Harness,再谈 Evolution,最后谈 Memory。任何团队如果跳过 Harness 直接做 Memory,会发现学到的全是噪声;跳过 Evolution 直接积累 Memory,会发现存了一堆但用不起来。

2.2 物理位置 × 演化深度的矩阵

第二个框架把两种常见的分层思路合并成一张矩阵。

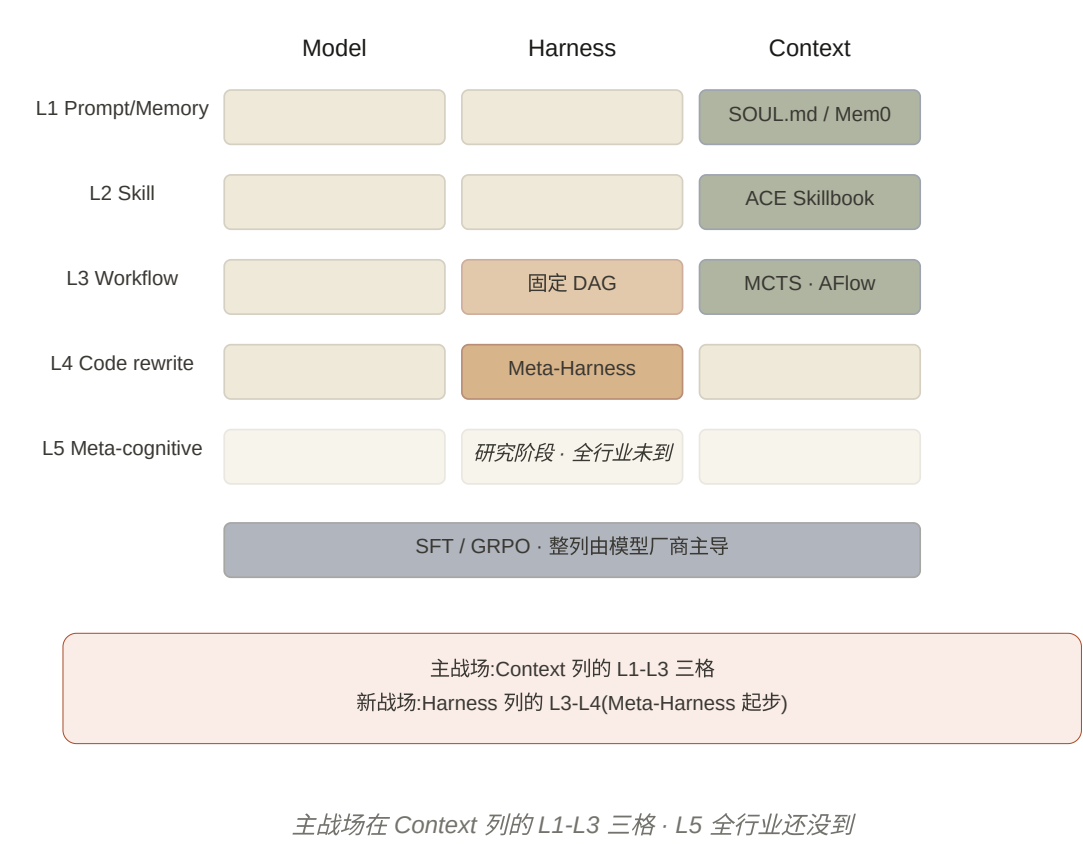
Harrison Chase (LangChain) 的三层(原文)按"物理位置"切:

- **Model** — 模型权重本身;通过 SFT、GRPO 等更新;灾难性遗忘是难题
- **Harness** — 核心循环代码 + always-on 工具;Meta-Harness 论文提出"coding agent 重写 agent 代码"
- **Context** — CLAUDE.md / SOUL.md / skills / memory;可以在 agent / user / team / org 四种粒度并存

另一种常见的分层 L1-L5 演化深度按"自我演化能力"切:

- L1 Prompt/Memory · L2 Skill auto-discovery · L3 Workflow/Topology · L4 Code self-rewriting · L5 Meta-cognitive

合并成一张 3×5 矩阵后,你会看到当前生态的真实分布:



这张矩阵给的三个判断

- A

tenant 级 Context 演化是 Day-1 能力,Agent 级全局演化是后置能力。Harrison 明确把 user/org/team 级更新摆到和 Agent 级同等重要的位置。这跟"Agent 全局自我演化"的叙事存在优先级翻转。对企业级产品,tenant 级才是 Day-1。
- B

对外讲故事用三层,对内决策用 L1-L5。三层简洁、好理解,适合传播;L1-L5 暴露演化深度差异,适合架构决策。两者正交而非对立。
- C

Harrison 的 Context 是个"大口袋",粒度不够。instructions、skills、tools、memory 全都混在 Context 一层。用 L1-L5 切开,能看到"更新一条用户偏好"和"让 agent 写出新 Skill 文件"在 Harrison 框架里都叫 Context 演化,但风险与价值天差地别。

2.3 三个框架的关系

本手册到这里出现了三个框架:三热点依赖链、Harrison 三层、L1-L5 演化深度。它们处理的是不同问题,可以这样定位:

框架	回答的问题	典型应用
三热点依赖链	"先做什么后做什么?"	团队路线图排序
Harrison 三层	"这东西存在哪?"	对外讲故事 / 产品架构图
L1-L5 深度	"这东西能自己改自己吗?"	内部架构决策 / 竞品分析

深度案例 · Memory Scaling

用 Databricks AI Research 在 2026-04-10 发表的 Memory Scaling for AI Agents 作为完整解剖样本。这是一篇文献,但本节不止于复述——它把博客内容融合到本手册的框架中,加上对照其他视角浮现的判断。

3.1 博客的三个核心论断

论断 A · 三条 scaling 轴并列

Databricks 明确把 memory 作为和 parametric / inference-time 并列的第三条 scaling 轴:

轴	手段	边际成本	红利归属
Parametric	模型做大、参数更多	高(算力+数据上限)	模型厂商
Inference-time	推理链路做深、多步反思	中(token 成本)	Agent 框架层
Memory Scaling	外部持久化状态累积	低(可摊薄)	企业自己

论断 B · Agent 身份的迁移

Agent 的身份(identity)正在从模型权重迁移到外部 state。当系统的 skill / knowledge / memory 足够充分时,更换底层 LLM 并不等同于丢失 Agent。新模型读取同一份持久化外部状态,立即继承所有累积经验。

这把 LLM 从"核心资产"降级为"可替换的推理引擎"。对架构决策有直接含义:任何对模型的定制(fine-tune 等)都应尽量推到 scaffolding 层去实现。

论断 C · 实验证据

指标	变化	含义
Test Accuracy(标注数据)	0% → 70%	超越 expert baseline 约 5%
Test Accuracy(无标注 user logs)	2.5% → 50%+	仅 62 条 log 就超过 expert,曲线更陡
Reasoning Steps	20 → ≈5	逼近硬编码理想 3.8 步

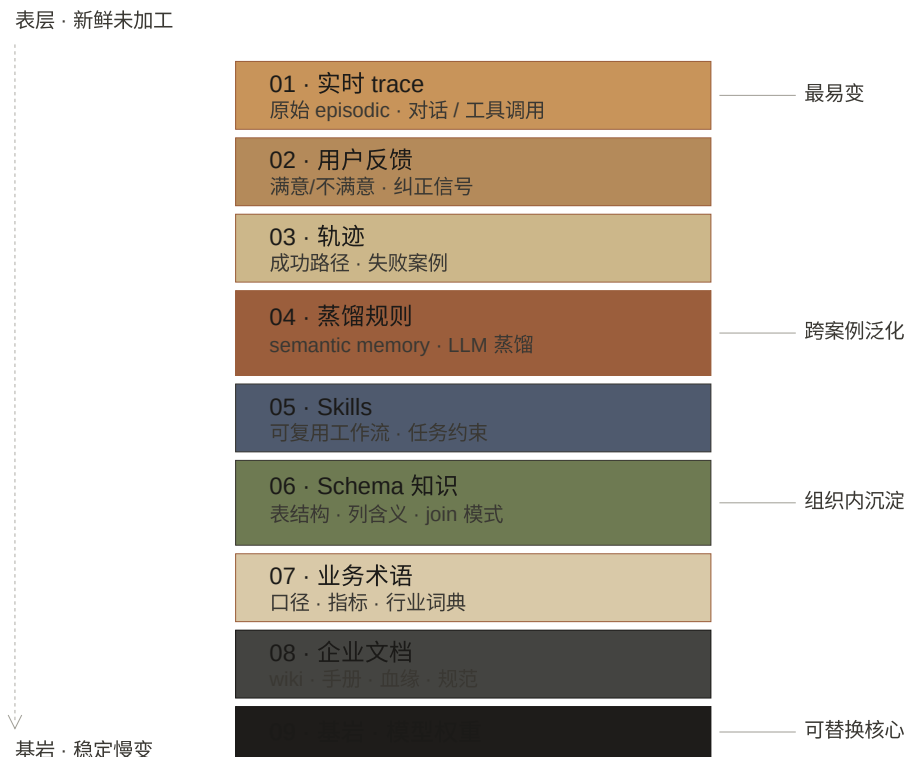
关键发现:未经人工标注的 user logs,只需 LLM judge 过滤,就能产生 scaling 曲线。产品的自然使用本身就是 memory engine 的数据源。

3.2 记忆的两个视角:结果 vs 过程

对 Memory Scaling 的完整理解需要两个互补视角:结果视角(记忆长成什么样)+ 过程视角(记忆怎么长出来)。

结果视角 · 九层地层

从表层新鲜的 episodic 到基岩稳定的 parametric,记忆按时间、加工程度、稳定性形成分层结构。



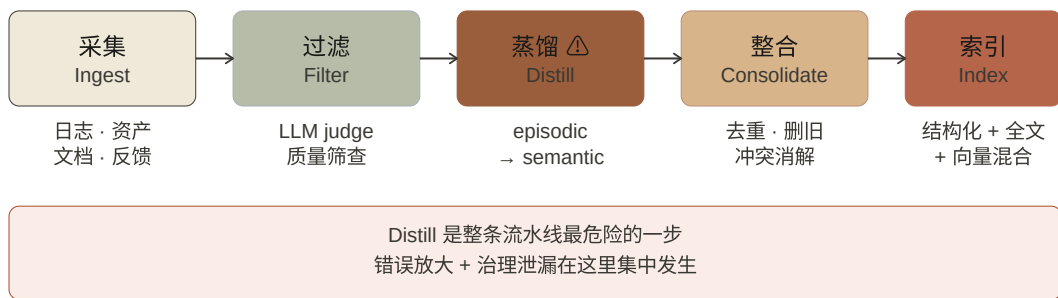
从实时 trace 到模型权重 · 越往下越稳定 · 也越敏感

三条设计约束:

- 存储策略必须分层。顶层易变 episodic 需要低延迟写入和高频检索;底层稳定 semantic 需要结构化索引和治理。单一 store 同时服务所有层会同时损失延迟和治理。
- 蒸馏是跨层运动。把反复出现的 trace 变成规则,把规则变成 skill。这是记忆从易变往稳定迁移的唯一路径。
- 每一层独立治理。越往下越稳定,但也越敏感——基岩层泄漏的后果和表层不可同日而语。治理标签必须穿透层与层之间的转化。

过程视角 · 五段加工流水线

地层回答"长什么样",流水线回答"怎么长出来"。从原始信号到可用记忆,中间有一条五段流水线。



Distill 是整条流水线最危险的一步 · 错误放大 + 治理泄漏在这里集中发生

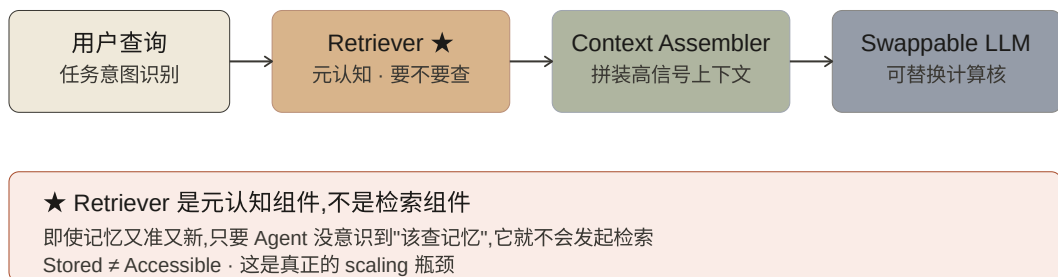
Distill 是整条流水线最被低估的环节,有两个致命特性:

- 错误放大:侥幸通过 Filter 的失败 trace,经过 Distill 升华成 semantic rule,从此被当作稳定知识反复调用。这是 memory pollution 的主路径。
- 治理泄漏:一条看似无害的 rule(如"A 公司经常 join CRM、市场情报、合作伙伴三张表")可能实际上在泄漏 A 公司的并购意图。治理属性必须在 Distill 环节被显式传递,不能假定它会自然继承。

设计原则:Distill 产生的每一条 semantic rule 都应该带上完整的 provenance(来自哪些 episodic trace)和 sensitivity label(至少继承上游最严格的标签)。

3.3 运行时:Stored ≠ Accessible

记忆"存在"不等于会被"用到"。真正的瓶颈在运行时。



Retriever 是元认知组件,不是检索组件 · 其质量决定 memory 能否被用到

这是博客最深刻的洞察。Retriever 不是普通的检索组件——它需要具备元认知能力:

- 知道什么时候该查(当前任务是否真的需要历史信息)

- 知道查哪一层(episodic vs semantic vs organizational)
- 知道用什么检索角度(关键词 / 语义 / 结构化)

因为 Stored knowledge 和 Accessible knowledge 之间的距离,就是由 Retriever 的元认知能力决定的。这一层的质量很可能比 memory store 本身的质量更决定 Agent 的最终表现。

3.4 治理与三道断层

企业级 memory 必须像数据资产一样被治理。这不是附属能力,而是 enterprise memory system 的前置条件。

治理四件套

1. Access Control(身份感知 ACL) — 一个人 / 团队 / 组织三层作用域必须强边界隔离,不能仅靠软约束
2. Lineage & Audit — 每一条记忆的出处、提炼路径、调用历史必须可追溯
3. Freshness & Retention — 时效信号、保留策略、周期性重验证。"高频检索的记忆会自信地过时"
4. Distillation 敏感性穿透 — 治理标签必须沿着 Distill 链路传递

三道断层(可预见的失败模式)

I 质量衰减。一次错误记忆被反复调用 → Agent 表现随时间退化而非提升。一个 stateless Agent 只会犯孤立错误,但一个 memory-equipped Agent 会把一次错误变成一个反复出现的错误。

II 治理漂移。蒸馏后的 semantic 泄漏原始敏感模式 → 合规风险、商业泄密。访问控制和敏感性标签必须在 ingestion 和 distillation 两个环节都被传递。

III 元认知盲点。Agent 不知道该调用哪段记忆 → 记忆累积但不被利用,投入低回报。这是真正的 scaling 瓶颈。

3.5 三个博客没触碰但企业必须追问的问题

- Q1** 记忆的理论上限在哪里?所有 scaling 曲线最终都会饱和。Databricks 暗示答案是"accessible 和 stored 的差距",也就是 Retriever 能力上限。这意味着投入足够存储后,边际回报会转向 retrieval 机制优化。
- Q2** memory scaling 和 inference scaling 是否替代?如果"检索替代探索"成立,对 LLM 推理深度的需求会下降。更小 LLM + 更厚 memory 可能比更大 LLM + 更薄 memory 更经济。这对成本结构有重大含义。
- Q3** 组织级 memory 的所有权归谁?当 memory 成为企业核心资产,SaaS 提供方的 memory 和企业自己的 memory 应该如何划分?这是商业层问题,但在企业采购决策里是关键变量。

判断与争议

所有"站得住脚的判断"汇总到这里,不再散落在前文各处。分两类:已确认的判断(带证据),悬而未决的问题(带追问方向)。

4.1 已确认的判断

关于宏观格局

- 01 内部机制(Harness / Evolution / Memory)全行业都在追,差异化靠外部基建三选一(RL Env / Context Engineering / Memory OS)。谁先在外部基建中找到自己的支点,谁就先建立结构性优势。
- 02 Harness 正在长出"数据可信层"。所有 orchestration 框架都不管数据进入 context 前的可信度。OWASP 把 memory poisoning 列为最关键 agent 安全风险。证据:Atlan 等数据治理厂商正在切入这块空白。
- 03 Self-Evolution 正在合流到 Context Engineering。证据:ACE 论文 + Skillbook 开源 + 北大 Meta Context Engineering 三者合流。"我是第一个融合 Memory + Skill 的"差异化窗口正在关闭。
- 04 "更小模型 + 更厚 scaffolding"已经能打败"更大模型 + 更薄 scaffolding"。证据:DeepSeek-V3.1 + ACE 在 AppWorld 榜单上匹配 GPT-4.1 + IBM CUGA。

关于架构决策

- 05 tenant 级 Context 演化是 Day-1 能力,Agent 级全局演化是后置能力。这个优先级不要翻转。Harrison Chase 的三层模型明确支持这个判断。
- 06 物理位置(Harrison 三层)× 演化深度(L1-L5)= 两个正交维度。对外讲用三层,对内决策用 L1-L5。
- 07 Retriever 不是检索组件,是元认知组件。它的质量决定 memory 能否被用到。Stored ≠ Accessible 是 memory scaling 的真正瓶颈。

08 Distill 环节必须显式传递治理标签,不要假定它会自然继承。这是 memory 治理最薄弱的一环。provenance + sensitivity label 是每一条 semantic rule 的最低标配。

09 单一 vector store 不够用。结构化 + 全文 + 向量的混合引擎是企业级 memory 的起点。Postgres 派系(Lakebase、Neon)天然支持事务、血缘、branching,更适合企业场景。

关于方法论与表达

10 一篇好博客值得两套视觉解读 — 工程架构图(用于评审)+ 情感叙事图(用于传播)。两者不是替代,而是覆盖不同场景。

11 图标不是意义的加速器。23 个 Lucide 图标会让图变成噪声。用排版和颜色编码语义,比图标更有效。

12 AI 之间的对比可以暴露盲点。让不同模型表达同一主题,然后对比差异,比自省更有效。*证据: Claude 版海报的"加工流水线缺失"是靠 ChatGPT 版的对比才看出来的。*

4.2 悬而未决的问题

Q1 Memory Scaling 的理论上限是什么?Databricks 暗示是 accessible 和 stored 的差距,但没给出具体界限。需要更多产线数据来验证。

Q2 memory scaling 和 inference scaling 是否存在替代关系?如果是,对 LLM 推理深度的需求会下降,模型选型策略会发生重大变化。

Q3 组织级 memory 的所有权、可迁移性、数据出境合规如何划分?这是商业层问题,但在企业采购决策里是关键变量。

Q4 ACE 这类"可演化 context"在生产系统中的长期 staleness 如何治理?论文给出的是单次实验数据,长期运行的退化曲线还需要观察。

Q5 L4 Code self-rewriting 在生产系统的现实路径是什么?Meta-Harness 论文给出了原型,但工程化路径(变更审计、回滚、渐进发布)还没成型。

资产清单 · 引用索引 · 方法论附录

5.1 引用索引

论文 / Paper

- ACE (Agentic Context Engineering) — arXiv 2510.04618 · Stanford + SambaNova · ICLR 2026
- Meta Context Engineering via Agentic Skill Evolution — 北大 2026
- Memory Scaling for AI Agents — Databricks AI Research · 2026-04-10
- Continual Learning for AI Agents — Harrison Chase (LangChain) · 2026-04-05
- Meta-Harness: End-to-End Optimization of Model Harnesses — Yoonho Lee
- MemOS: A Memory OS for AI System — arXiv 2507.xxxxx

基准 / Benchmark

- LOCOMO — memory 架构可复现对比的第一个基准
- AppWorld — ReAct + ACE 匹配 IBM CUGA 的场景
- tau2-bench — Sierra Research · ACE 一致性翻倍测试

框架 / 工具

- ACE Framework — github.com/ace-agent/ace; github.com/kayba-ai/agentic-context-engine
- OpenReward / ORS Protocol — 330+ RL 环境作为 API 服务
- HUD / Prime Intellect / Scale RL Environments — RL 环境基建

产品 / 平台

- Lakebase(Databricks · Neon 引擎)— serverless Postgres for AI agents
- Mem0 — 覆盖 21 个框架的 memory 层
- Atlan — Gartner Leader 2026 · Data Governance

5.2 视觉资产清单

资产	用途	位置
memory_scaling_sedimentary.pdf/png	A3 博物馆级海报 · 地质风格 · 6 子图 覆盖博客骨架	本手册随 附文件
memory_scaling_design_philosophy.md	Sedimentary Intelligence 设计哲学	本手册随 附文件
memory_scaling_fusion_v3.md	融合版给 AI 消费的完整架构文档	本手册随 附文件

5.3 方法论附录 · 关于"如何用对话推进认知"

本手册沉淀的内容来自一段五轮的对话讨论。结构上它已经被重新组织,但思考过程本身也值得记录。

方法 1 · 先粗后细

讨论从用户提出三个概念名开始,不急着给架构。先确认宏观方向,再下钻具体文献。这避免了"在错的方向上做精致工作"。

方法 2 · 对比是暴露盲点的最快方法

如果只让一个模型做可视化,你永远不知道它漏了什么。让两个模型做同一题,差异本身就是认知增量。本手册第 3 部分对加工流水线的强调,完全来自这一对比。

方法 3 · 沉淀按知识层级,不按对话时间

第一版沉淀文档按"对话五轮"组织,但读者要自己拼出主题。这一版按"立场 → 框架 → 案例 → 判断 → 索引"的知识层级重组。同一主题的所有讨论合并到一处,不再被对话切碎。

方法 4 · 把悬而未决的问题单独列出来

成熟文档不怕暴露未知,只怕装作什么都懂。Part 4.2 的 Q1-Q5 是这份文档留给未来的种子,也是衡量未来认知更新的锚点。